

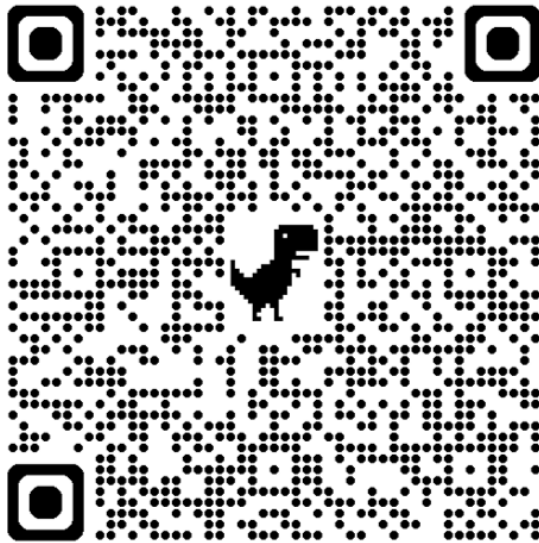
Intro to High-Performance Computing

Workshop Lead: Megan Ng

July 28, 2026

Mission : aims to deliver inter-disciplinary research programs and empower the use of data in health research and health care delivery

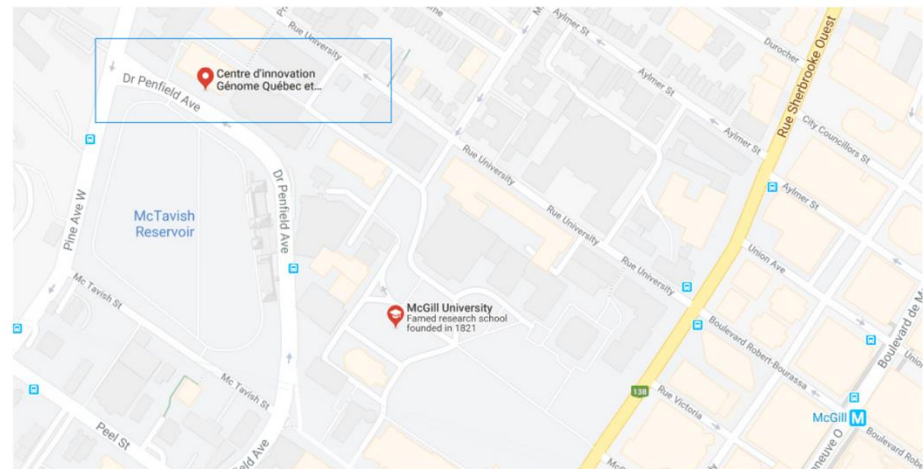
Scan for feedback!



<https://www.mcgill.ca/micm>

McGill.CA / MCGILL INITIATIVE IN COMPUTATIONAL MEDICINE

Contact

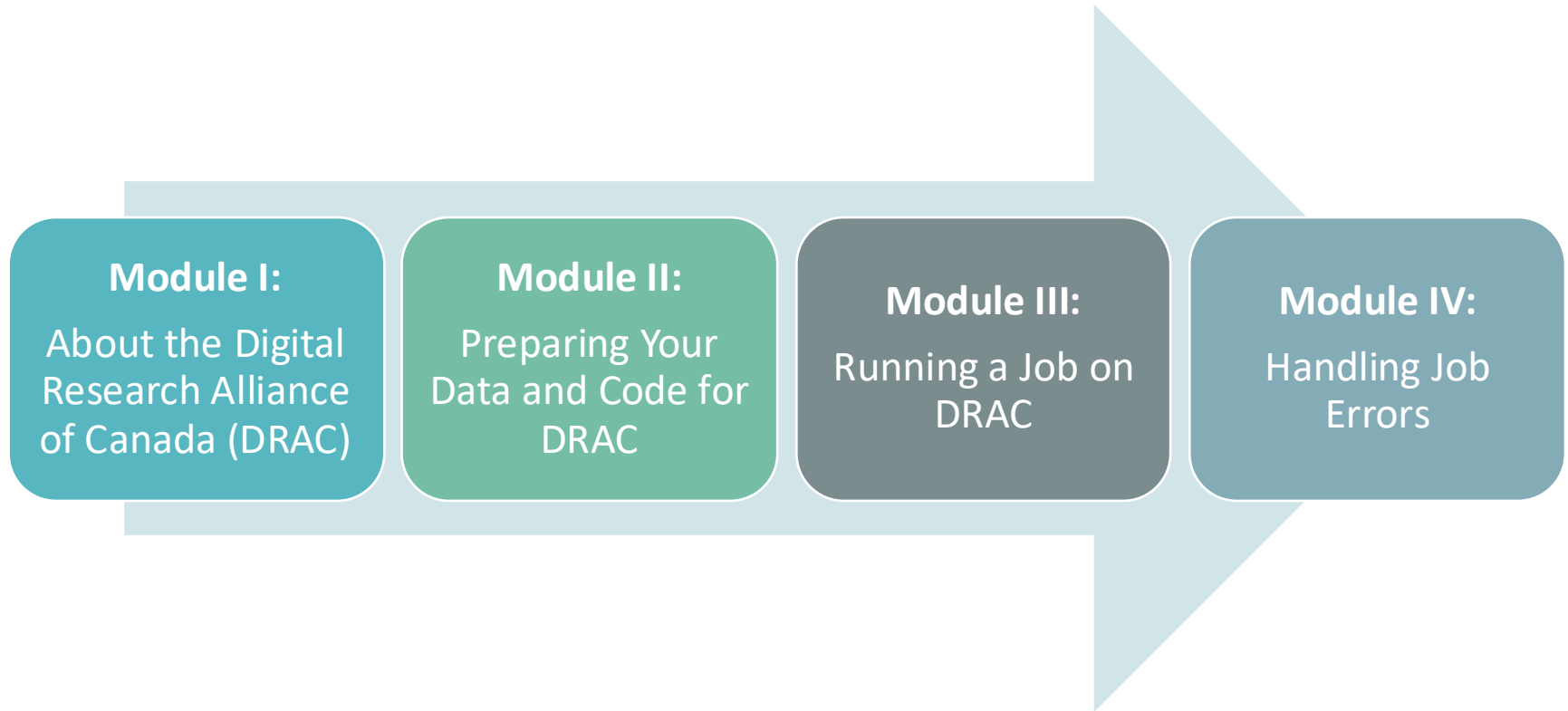


MiCoM McGill initiative in
Computational Medicine

McGill initiative in Computational Medicine
740, Dr. Penfield Avenue, Montreal, Quebec,
Canada, H3A 0G1
email: info-micm@mcgill.ca

[Signup](#) to our newsletter to receive the latest news

Workshop Outline



Pre-requisites

You must have...

- A terminal
- The Duo Mobile application downloaded on your phone
- A Digital Research Alliance of Canada account and access to one of their general-purpose clusters (Fir, Narval, Nibi, or Rorqual)
- Globus Connect Personal installed and running in the background on your laptop
- IntroToHighPerformanceComputing GitHub repository materials downloaded on your laptop
- A code editor (I will be using VS code)

Module I:

About the Digital Research Alliance of Canada (DRAC)

About DRAC

What is DRAC?

- A national not-for-profit membership-based organization that provides digital research services for researchers in Canada



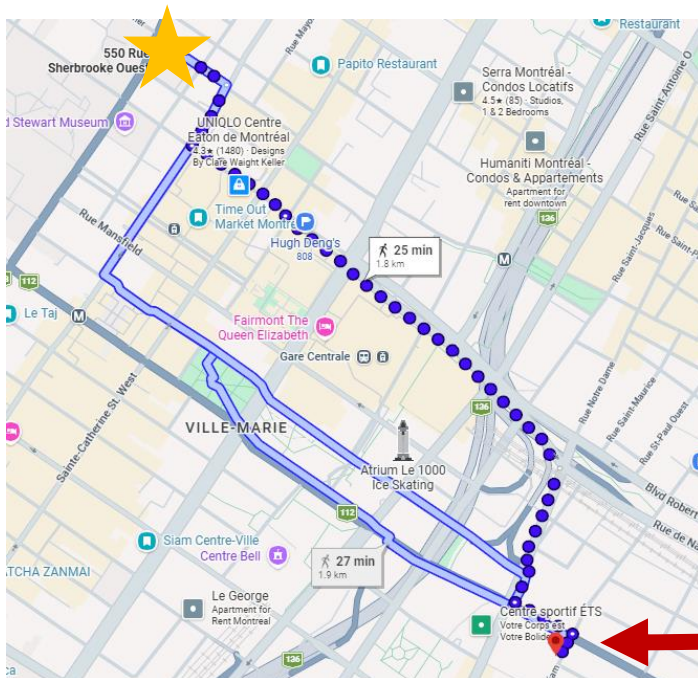
**Digital Research
Alliance** of Canada

Compute tamIA TamIA is a cluster dedicated to AI for Canadian researchers. Designed for workloads that... Read More >	Compute Trillium Trillium is a large parallel cluster comprising 240K CPU cores, a dedicated GPU... Read More >
Research Data Management DataCite Canada Consortium The DataCite Canada Consortium (DCAN) is a collective of organizations th... Read More >	Research Data Management DMP Assistant DMP Assistant is a national, online, bilingual tool developed by the Digital... Read More >
Research Data Management ORCID-CA ORCID-CA is the ORCID Canada Consortium. ORCID-CA provides Canadian...	Research Software CVMFS The CERN Virtual Machine File System (CVMFS) is a distributed read-only content...

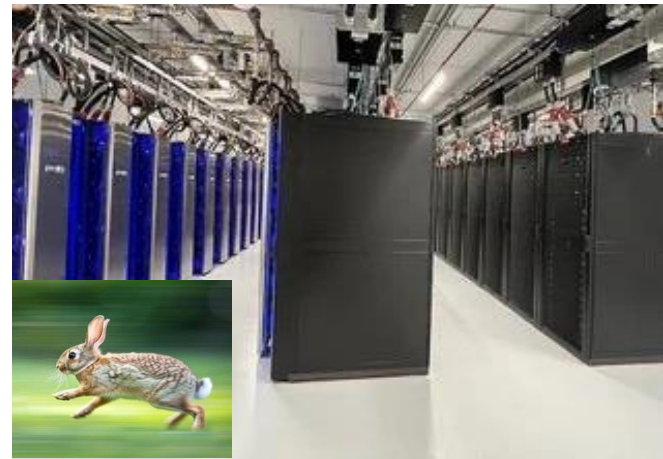
About DRAC

Why do we use DRAC?

- CPU/GPU and memory constraints can make computationally intensive tasks slow or impossible on your everyday computer



A DRAC Supercomputer



You and DRAC: An Analogy

Task: Make 2026 frozen dumplings by the end of 2026 (for good luck in 2027 duh)

Your Limitations:

SPEED

- You are one person → You would need to make 90 dumplings every single weekend

STORAGE

- You have a regular sized fridge/freezer → You do not have the space to store the ingredients nor completed frozen dumplings



You and DRAC: An Analogy

Distributed Recipe Acceleration Centre (DRAC):

- Cooking service that addresses your limitations
 - 100,000+ chefs
 - Large scale kitchen equipment/storage
- Can complete your cooking task efficiently



DRAC Service Rules

1. You must prepare and deliver your ingredients and recipe yourself
2. To schedule a cooking job, you must submit a job request to the DRAC manager
3. When the job is finished, you must pick up the final product yourself



You and DRAC Analogy Quiz

- 
1. You and your regular kitchen
2. Ingredients
3. Recipe
4. Distributed Recipe Acceleration Centre
- A. Digital Research Alliance of Canada
- B. Code
- C. Input data
- D. You and your regular computer

Module II:

Preparing Your Data and Code for DRAC

DRAC Service Rule #1

You must prepare and deliver your **ingredients** and **recipe** yourself



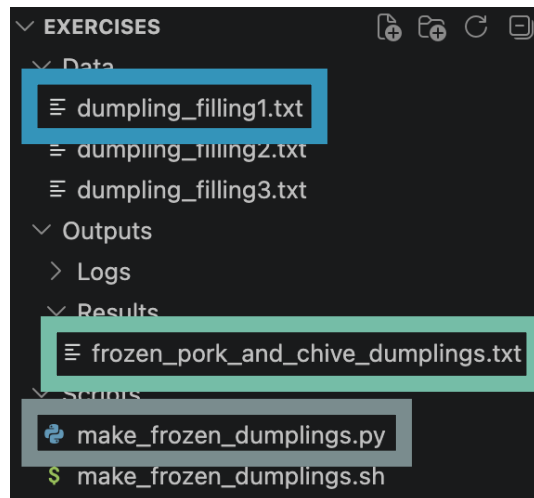
You must prepare and deliver your input **data** and **code** yourself



Step 1a: Preparing your data and code

Input Data

```
≡ dumpling_filling1.txt X  
data > ≡ dumpling_filling1.txt  
1    pork  
2    chive
```



Result

```
≡ frozen_pork_and_chive_dumplings.txt X  
Outputs > Results > ≡ frozen_pork_and_chive_dumplings.txt  
1  
2  
3  
4      FROZEN DUMPLINGS  
5      PORK & CHIVE  
6      ^^^ ^^^ ^^^ ^^^  
7      ( ) ( ) ( ) ( )  
8  
9      ^^^ ^^^ ^^^ ^^^  
10     ( ) ( ) ( ) ( )  
11  
12     KEEP FROZEN  
13     NET WT. 750 g  
14  
15  
16  
17      FROZEN DUMPLINGS  
18      PORK & CHIVE  
19      ^^^ ^^^ ^^^ ^^^  
20     ( ) ( ) ( ) ( )  
21
```

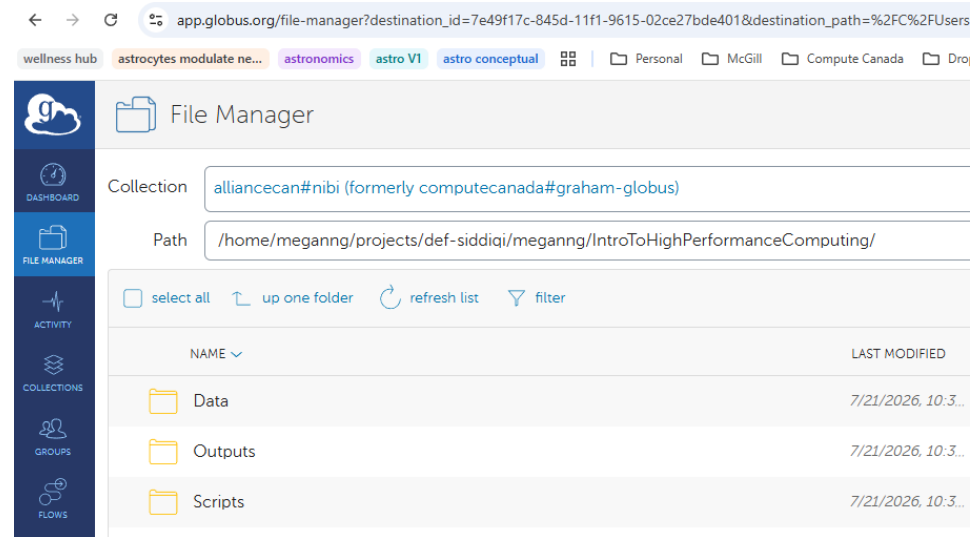
Code

```
def make_frozen_dumplings(input_file, output_folder, n_bags=3):
```

Step 1b: Delivering your data and code (with Globus)

About Globus:

- Web app for file management
- Use can use it to...
 - Visualize your files and folders
 - Transfer local files to and from DRAC



```
[meganng@l3.nibi IntroToHighPerformanceComputing]$ pwd
/home/meganng/projects/def-siddiqi/meganng/IntroToHighPerformanceComputing
[meganng@l3.nibi IntroToHighPerformanceComputing]$ ls
Data  Outputs  Scripts
```

Demo:

Navigating folders with linux commands

Activity: Get familiar with linux commands

1. What directory are you currently in?
2. Navigate to the "IntroToHighPerformanceComputing" folder
3. Navigate to the "Data" folder
4. Print the contents of the one of the files in the Data folder.
5. Execute the command "cd ..". Where does that take you?

Command	Description	Usage
pwd	Print current directory path	pwd
cd	Change directory	cd some/folder/path
ls	List contents of a folder	ls some/folder/path
cat	Print contents of a file	cat someFile.txt
mkdir	Makes a new directory	mkdir DirectoryName

Demo:

Creating a destination folder on DRAC with Globus

Activity: Create a destination folder on DRAC with Globus

1. Go to [Globus](#)
2. Click file manager
3. Search for the DRAC cluster you are using
4. Navigate to your projects folder
5. Create a new folder called
“IntroToHighPerformanceComputing”

Demo:

Logging into DRAC

Activity: Log into DRAC

1. In terminal
 - `$ ssh <username>@nibi.alliancecan.ca`
2. Enter your password
3. Enter “1” to receive Duo Push notification on phone
4. Accept Duo Push notification on phone
5. Navigate to new folder to verify it has been created

Demo:

Delivering files to DRAC with Globus

Activity: Delivering files to DRAC with Globus

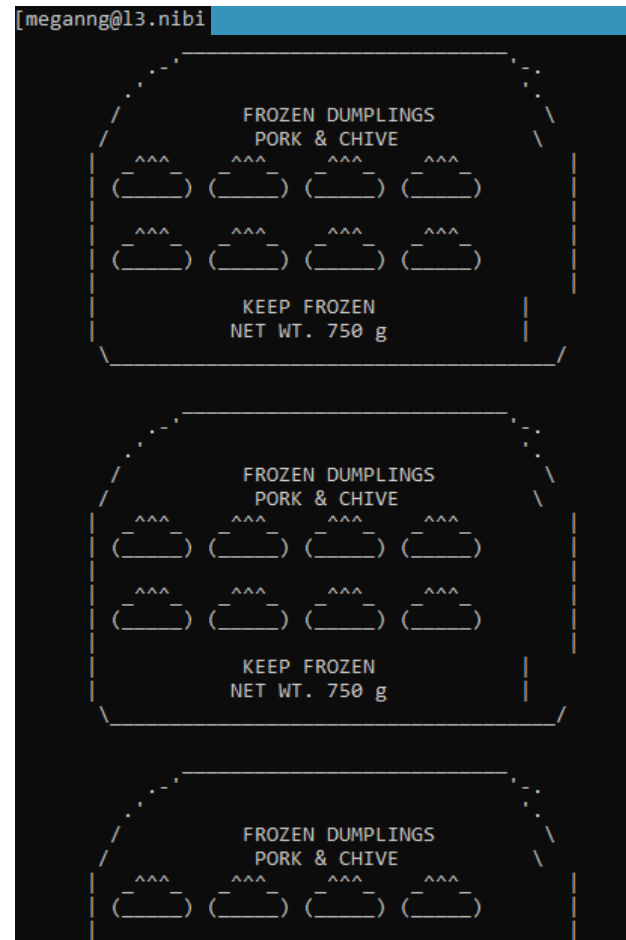
1. On Globus, set your destination folder to be “IntroToHighPerformanceComputing”
2. Click “Transfer or Sync to...”
3. Set your source folder to be the “IntroToHighPerformanceComputing/Exercises” folder on your local computer
4. Select all folders inside the “Exercises” folder
5. Click “Start”

Activity: Confirm your file transfer to DRAC

Goal: Print the contents of the file in the “Results” folder to terminal

Hint: You will need to use the following terminal commands

- `cd` → change directory
- `ls` → list contents of directory
- `cat` → print contents of file to terminal



Module III:

Running a Job on DRAC

DRAC Service Rule #2

To schedule a **cooking job**,
you must submit a job
request to **the DRAC
manager**



To schedule **your code
to run**, you must
submit a job to **slurm**



What is slurm?

- A job scheduler used by supercomputers
- Manages allocating compute resources among all users



Digital Research
Alliance of Canada



How do I submit a job to slurm?

1. Create a **batch script** (instructions for how to run your code)

```
$ make_frozen_dumplings.sh X
Scripts > $ make_frozen_dumplings.sh
1  #!/bin/bash
2  #SBATCH --account=def-siddiqi
3  #SBATCH --time=00:10:00
4  #SBATCH --mem=1G
5
```

} MANDATORY

How do I submit a job to slurm?

2. Submit your **batch script** with sbatch

- Usage → `$ sbatch batchScript.sh`



```
$ make_frozen_dumplings.sh X
Scripts > $ make_frozen_dumplings.sh
1  #!/bin/bash
2  #SBATCH --account=def-siddiqi
3  #SBATCH --time=00:10:00
4  #SBATCH --mem=1G
5
6  #SBATCH --chdir=/home/meganng/projects/def-siddiqi/meganng/IntroToHighPerformanceComputing/
7  #SBATCH --output=Outputs/Logs/dumplings_%j.out
8
9  #SBATCH --mail-user=megan.ng@mail.mcgill.ca
10 #SBATCH --mail-type=ALL
11
12 # Define all paths
13 script_path="Scripts/make_frozen_dumplings.py"
14 input_data="Data/dumpling_filling1.txt"
15 output_folder="Outputs/Results/"
16
17 # Load Python module
18 module load python/3.11
19
20 # Run the Python script with the input and output paths
21 python3 "${script_path}" "${input_data}" "${output_folder}"
```

Common slurm Commands

squeue / sq: Prints out information about your job submission (where you are in the queue, how long your job has been running, etc.)

- Usage → `$ sq`

```
[meganng@l3.nibi Scripts]$ sq
```

JOBID	USER	ACCOUNT	NAME	ST	TIME_LEFT	NODES	CPUS	TRES_PER_N	MIN_MEM	NODELIST	(REASON)
18142414	meganng	def-siddiqi_cpu	make_frozen_du	PD	10:00	1	1	N/A	5G		(Priority)

```
[meganng@l4.nibi Logs]$ sq
```

JOBID	USER	ACCOUNT	NAME	ST	TIME_LEFT	NODES	CPUS	TRES_PER_N	MIN_MEM	NODELIST	(REASON)
18476880	meganng	def-siddiqi_cpu	make_frozen_du	R	4:15	1	1	N/A	5G	c491	(None)

scancel: Cancels a submitted job

- Usage → `$ scancel [jobID]`

Common slurm Commands

seff: Prints out information about a completed job (ex. How much time/memory it took in total)

- Usage → `$ seff [jobID]`

```
[meganng@l3.nibi Logs]$ seff 18161373
Job ID: 18161373
Cluster: nibi
User/Group: meganng/meganng
State: OUT_OF_MEMORY (exit code 0)
Cores: 1
CPU Utilized: 00:00:00
CPU Efficiency: 0.00% of 00:00:02 core-walltime
Job Wall-clock time: 00:00:02
Memory Utilized: 997.89 MB
Memory Efficiency: 97.45% of 1.00 GB (1.00 GB/node)
```

Activity: Submit your job to DRAC

1. On DRAC print out the contents of the current batch script in the “Scripts” folder
2. On your local computer, modify the batch script
 - Personalize lines ending with “# CHANGE”
 - Change the input data text file to filling2 or filling3
3. Upload it to DRAC with Globus
4. On DRAC, print out the contents of the updated batch script to confirm your changes have been uploaded
5. Submit your batch script with sbatch
6. Check on the job’s status with squeue

Module IV:

Handling Job Errors

Output and Error Files

Output File: The output of any print statements within your batch script and code will be found here

Error File: Any error messages generated during your job will be found here

```
$ make_frozen_dumplings.sh X
Scripts > $ make_frozen_dumplings.sh
1  #!/bin/bash
2  #SBATCH --account=def-siddiqi
3  #SBATCH --time=00:10:00
4  #SBATCH --mem=1G
5
6  #SBATCH --email=nibibio@mcgill.ca
7  #SBATCH --output=Outputs/Logs/dumplings_%j.out
8  #SBATCH --error=Outputs/Logs/dumplings_%j.err
9
10 #SBATCH --mail-user=megan.ng@mail.mcgill.ca
11 #SBATCH --mail-type=ALL
12
13 # Define all paths
14 script_path="Scripts/make_frozen_dumplings.py"
15 input_data="Data/dumpling_filling1.txt"
16 output_folder="Outputs/Results/"
```

```
[meganng@l3.nibi Logs]$ ls
dumplings_18161373.err  dumplings_18161373.out
```

Common Errors and Their Solutions

Error Type	Error Signal	Solution
Insufficient time or memory requested	Error message written in output/error file and/or email	Adjust time or memory request in batch script
Incorrect path to file	Error message written in output/error file and/or email	Correct path in batch script
Missing libraries/modules/packages	Error message written in output/error file and/or email	Load in necessary libraries/modules/packages in batch script

How to Debug on DRAC:

1. Identify error by reading output/error file and/or email
2. Edit batch script accordingly

General Advice

- Make sure your actual code is bug free before running it on DRAC
 - Eases DRAC debugging process by limiting sources of possible errors
- Request only the time and memory you NEED
 - More demanding jobs usually == longer time before your job begins
- If you are experiencing an unexplainable error, first check the status of the cluster you are using
 - Google “DRAC [cluster name] status”

DRAC Technical Support

- They are friendly and fast!
- See https://docs.alliancecan.ca/wiki/Technical_support for more info

Please choose the address that corresponds best to your question or issue:

- accounts@tech.alliancecan.ca -- Questions about accounts
- renewals@tech.alliancecan.ca -- Questions about account renewals
- globus@tech.alliancecan.ca -- Questions about **Globus** file transfer services
- cloud@tech.alliancecan.ca -- Questions about using **Cloud** resources
- allocations@tech.alliancecan.ca -- Questions about the [Resource Allocation Competition](#) (RAC)
- trillium@tech.alliancecan.ca -- For questions or issues regarding to the **Trillium** cluster specifically
- support@tech.alliancecan.ca -- For any other question or issue

Activity: Debug your batch script

1. Print out the contents of the .out file. What type of error occurred?
2. Fix the error in your batch script, update the file locally on your laptop, sync the changes you made to DRAC with Globus, and resubmit the job with sbatch.
3. Oh no there is another error... repeat steps 1-2

Demo:

Retrieving your results from DRAC with Globus

Activity: Retrieve your results from DRAC with Globus

1. Go to [Globus](#)
2. Click file manager
3. Search for the DRAC cluster you are using
4. Navigate to your “Results” folder
5. Select the resulting text file you want to retrieve
6. Click “Transfer or Sync to...”
7. Search for your local computer and navigate to your “Results” folder
8. Click “Start”
9. Open the retrieved text file on your laptop to verify the transfer

Summary

Module I:

- What is DRAC and the services they provide
- The conceptual workflow of interacting with DRAC

Module II:

- How to upload files from your local computer to DRAC
- How to download files from DRAC to your local computer

Module III:

- How to run your code using DRAC's high-performance computing system
- How to monitor the status of your job

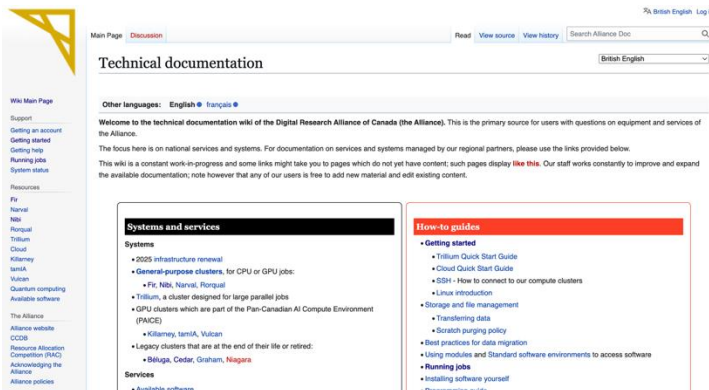
Module IV:

- How to identify and address common errors
- How to get external technical help when needed

Additional Information

DRAC's Wiki Page:

- All information not covered in this workshop can be found here
- When you encounter an issue, try searching for the solution here first
- This website is informative but dense → Do not hesitate to reach out to technical support if you feel stuck



Installing packages/libraries:

- Your own code likely has dependencies from various packages/libraries
- The protocol for making these dependencies accessible to your code when a job is run varies slightly between programming languages
- All information regarding these protocols can be found on DRAC's wiki page

Creating virtual environments inside of your jobs

Note: On Trillium it is recommended to create virtual environments from a log

Parallel filesystems such as the ones used on our clusters are very good at r loading libraries, such as starting Python and loading a virtual environment, c

As a workaround for this kind of slowdown, and especially for single-node Py seem counter-intuitive to recreate your environment for every job, but it can t performance issues. This approach, of creating a node-local virtualenv, has b script demonstrates how to do this for a single-node job:

```
File : submit_venv.sh

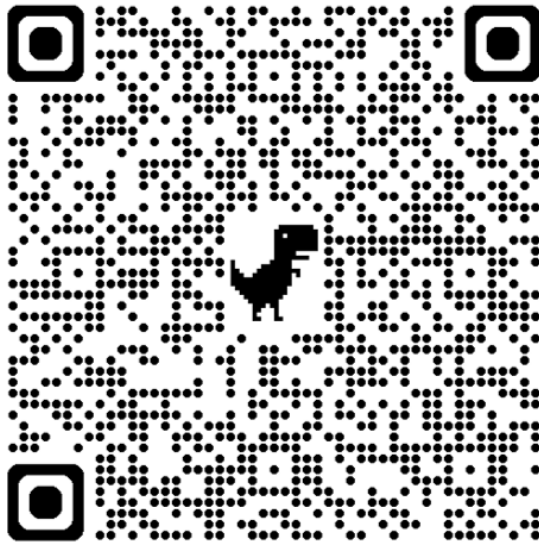
#!/bin/bash
#SBATCH --account=def-someuser
#SBATCH --mem-per-cpu=1500M # increase as needed
#SBATCH --time=1:00:00

module load python/3.11
virtualenv --no-download $SLURM_TMPDIR/env
source $SLURM_TMPDIR/env/bin/activate
pip install --no-index --upgrade pip

pip install --no-index -r requirements.txt
python ...
```

Mission : aims to deliver inter-disciplinary research programs and empower the use of data in health research and health care delivery

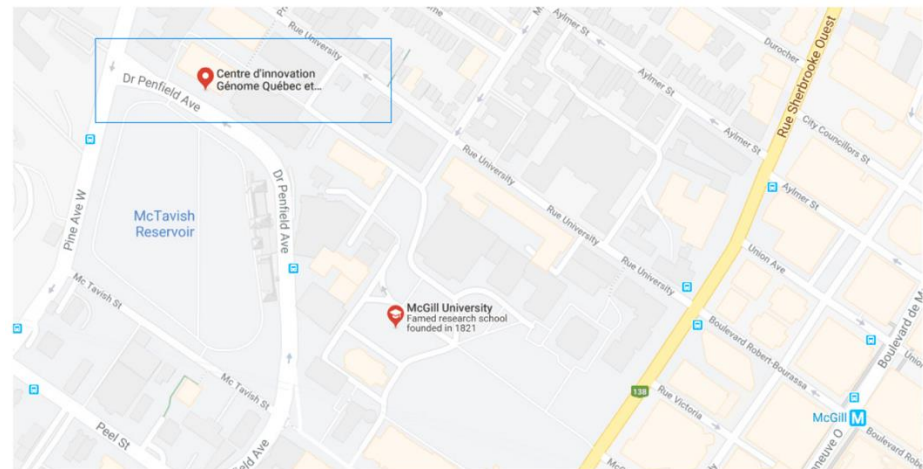
Scan for feedback!



<https://www.mcgill.ca/micm>

McGill.CA / MCGILL INITIATIVE IN COMPUTATIONAL MEDICINE

Contact



MiCoM McGill initiative in
Computational Medicine

McGill initiative in Computational Medicine
740, Dr. Penfield Avenue, Montreal, Quebec,
Canada, H3A 0G1
email: info-micm@mcgill.ca

[Signup](#) to our newsletter to receive the latest news